# Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #6

CS@Cluj-Napoca

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# Agenda

- **Trees (BST and regular trees)**
  - insert
  - search
  - delete
- **Incomplete structures**
  - meaning/utility
  - lists
  - trees – next lecture

# Insert in BST

```
%insert_bst/3(In_tree, Key_to_ins, Out_tree).
```

**Q:** is the order of arguments relevant ? Why/why not?

```
insert_bst(nil, K, t(nil,K,nil)):-!.  % insert_bst(T, K, R):-T=nil,!, R = t(nil,K,nil).
insert_bst(t(Left,K,Right), K, t(Left,K,Right)):-!,
        write("in tree").
insert_bst(t(Left,Key,Right), K, t(NewLeft,Key,Right)):-
        K<Key,!,
        insert_bst(Left,K,NewLeft).
insert_bst(t(Left,Key,Right), K, t(Left,Key,NewRight)):-
        insert_bst(Right,K,NewRight).
```

**Qs:** **1.** Where is inserted (position)? ALWAYS LEAF! NO EXCEPTION!
**2.** What does ! in clause 1 negate? In clause 2?

**Time estimate:** a=1, b=2 (if balanced), c=0 =>O(lgn)

**Q:** Estimate best/worst case (situation) and time

# Insert in regular (not search) tree

```
%insert/3(In_tree, Key_to_ins, Out_tree).

insert(nil, K, t(nil,K,nil)):-!.  % insert(Tin, K, Tout):-Tin=nil, !, Tout=t(nil,Key,nil).
insert(t(Left,K,Right), K, t(Left,K,Right)):-  % insert(Tin,K,Tout):-
       !, write("in tree").  % Tin=t(Left,K,Right),!,write("in tree"),Tout= t(Left,K,Right)).
insert(t(Left,Key,Right), K, t(NewLeft,Key,Right)):-
       insert(Left,  K, NewLeft).
insert(t(Left,Key,Right), K, t(Left,Key,NewRight)):-
       insert(Right, K, NewRight).
```

- **Qs:**
  - **1.** Where is inserted (position)? Specifically!
  - Oral discussion on OR nondeterminism for parallel/concurrent processing.

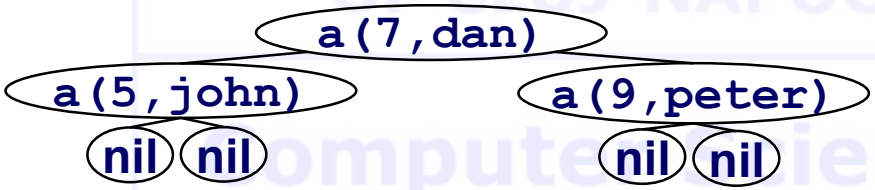
# Search in BST

Assume the node contains a structure of type a(Key,Value)
The tree is BST based on key

Ex:

```
tree(
    n(
        n(nil, a(5,john), nil),
                                a(7,dan),
        n(nil, a(9,peter), nil)
    )
).
```

Represents a tree with 7 and 2 leaves (7 and 9).

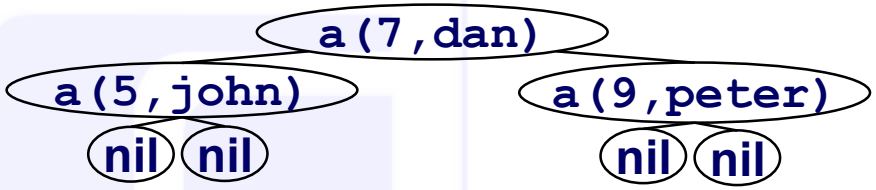



If stored in knowledge base, processing is with:

```
?-tree(T), process(T,…).
```

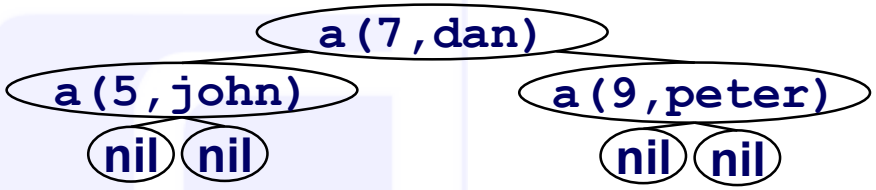
# Search in BST (search by Key)

```
%is_in_stree/2(Node_searched,Tree)

is_in_stree(N, n(_,Node,_)):-
       eqch(N, Node),!,
       N=Node.
is_in_stree(N, n(Left,Node,_)):-
       ord(N, Node), !,
       is_in_stree(N, Left).
is_in_stree(N, n(_,_,Right)):-
       is_in_stree(N, Right).

eqch(a(K,_),a(Key,_)):-
       nonvar(K),
       K=Key,!.

ord(a(K,_),a(Key,_)):-
       nonvar(K),
       K<Key,!.
```



# Search in BST (search by Key) – contd.

```
eqch(a(K,_),a(Key,_)):-
        nonvar(K),
        K=Key,!.
```




- Predicate to check if the *key* we are *searching* for has the **same** value as the *key* in the *current node* in the tree.
- Qs:
  - What is the meaning of `nonvar(K)` and why is needed?
  - "!" is not mandatory! Why?

```
ord(a(K,_),a(Key,_)):-
        nonvar(K),
        K<Key,!.
```

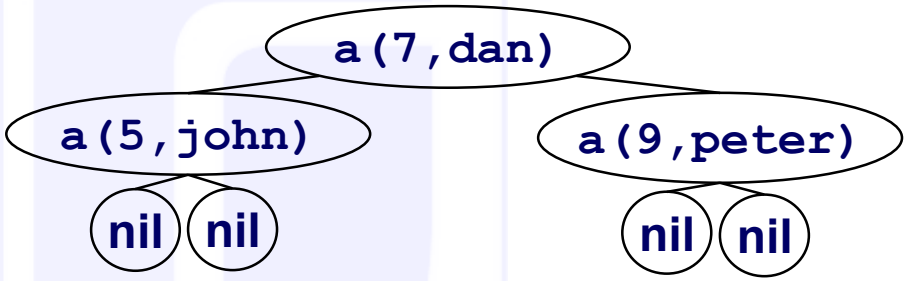
- Predicate to check if the key *K* we are *searching* for has a **smaller** value than the key *Key* in the *current node* in the tree.
- Qs:
  - What is the meaning of `nonvar(K)` and why is needed?
  - "!" is not mandatory! Why?

```
?-tree(T),is_in_stree(a(9,X),T).
```

Yes, T=..., X=...?

# Search in regular tree (search by Value)

```
%is_in_tree/2(Key_searched,Tree)

is_in_tree(N, n(_,N,_)).
is_in_tree(N, n(Left,_,_)):-
       is_in_tree(N, Left).
is_in_tree(N, n(_,_,Right)):-
       is_in_tree(N, Right).
```




```
?-tree(T),is_in_tree(a(R,john),T).
```

Yes, T=…, R=5.

%T would be the tree read, R the key assoc. to the value (name) provided

```
?-tree(T),is_in_tree(a(9,X),T).
```

% can we ask this way? What's the answer? What's the meaning?

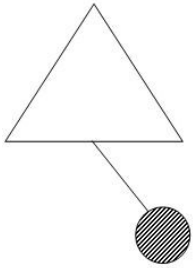
X=…?

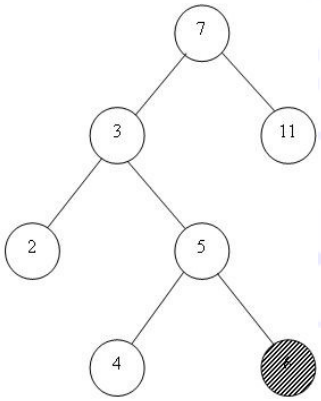
# Delete from BST

- Search for the node where the key resides + remove the node
- Delete a key = remove the node containing the key
- Cases to consider:
  - Leaf – just remove the node
  - One child node – shortcut the node (link the child to its grandparent)
  - Two children (⇔ delete the root after the search stage) different story

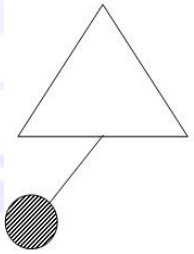
Figs. For removal of leaves

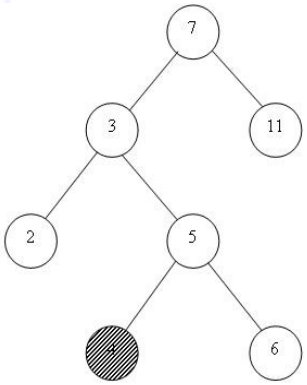
Case 1






Case 2

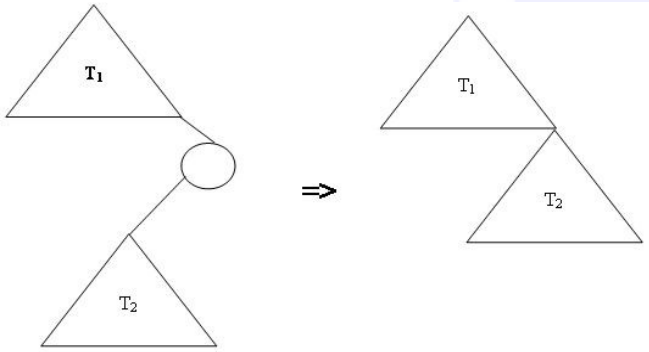


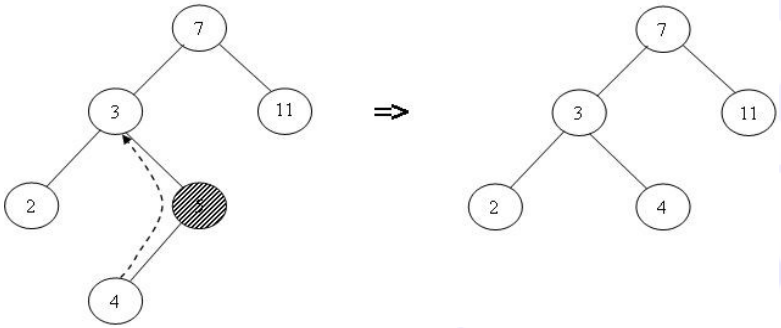


# Delete from BST – contd.

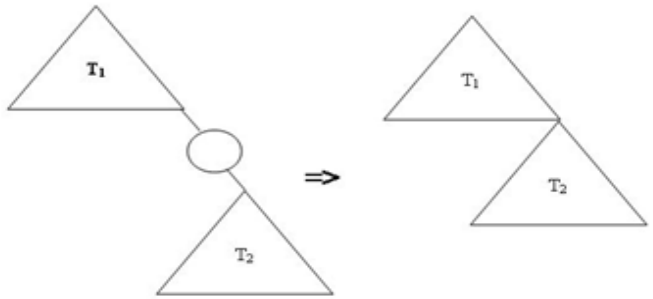
Removal of a right child node with just one subtree

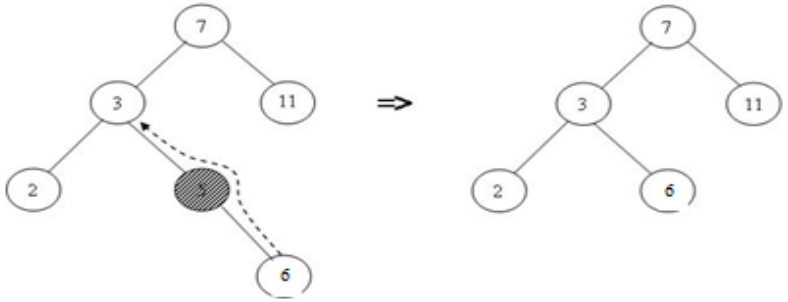
Case 3




Case 4

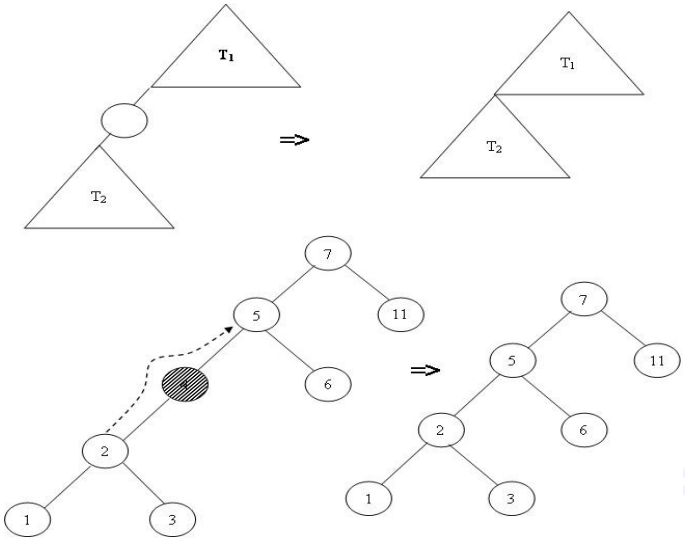


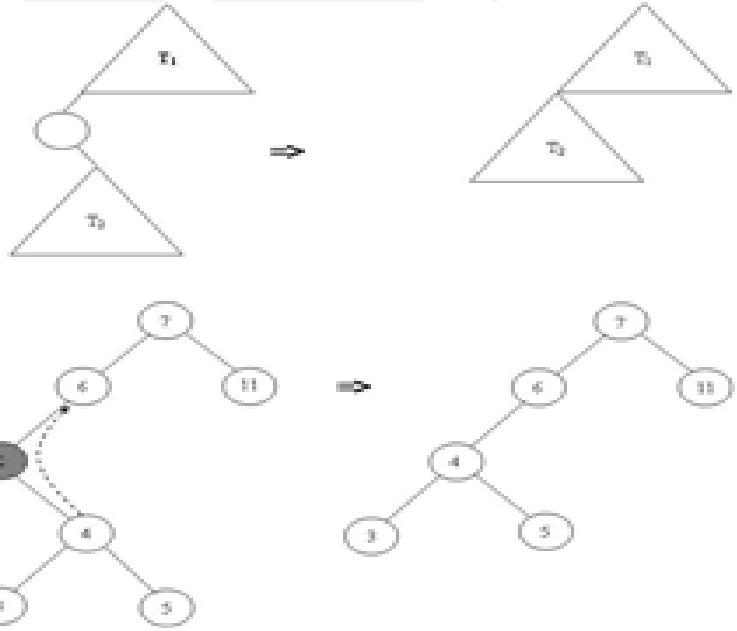
# Delete from BST – contd.

Removal of a left child node with just one subtree

Case 5




Case 6



- How about root? Postpone for the moment.
- Let's reach this point with code.

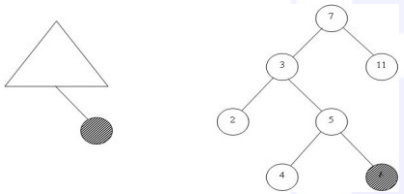
# Delete from BST - code

```
% delete_bst/3(In_tree, Key_to_del, Out_tree)

% search key; find the node to contain the key
delete_bst(t(Left,Key,Right),K,t(NewLeft,Key,Right)):-
       K<Key,!,
       delete_bst(Left,K,NewLeft).
delete_bst(t(Left,Key,Right),K,t(Left,Key,NewRight)):-
       K>Key,!,
       delete_bst(Right,Key,NewRight).

% found key; let's solve the easy cases discussed before
delete_bst(nil,      K, nil):-!,
       write(K), write('not in tree').
delete_bst(t(nil,  K, nil),    K, nil):-!.
delete_bst(t(nil,  K, Right), K, Right):-!.
delete_bst(t(Left, K, nil),    K, Left):-!.

% "difficult" case postponed
```

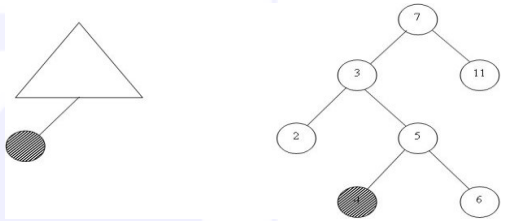
TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# Delete cases

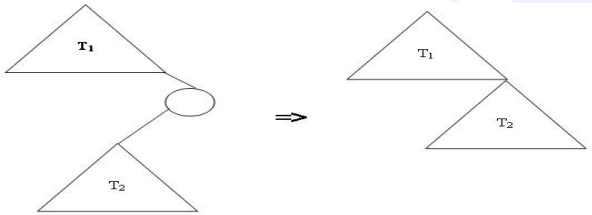
1



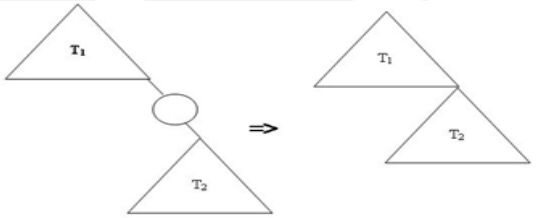
2

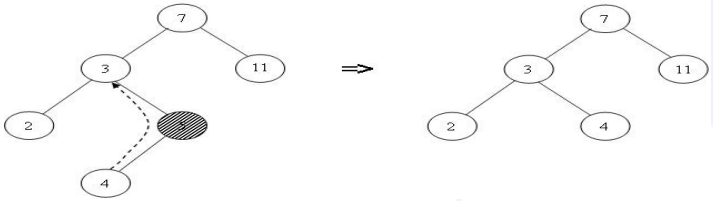


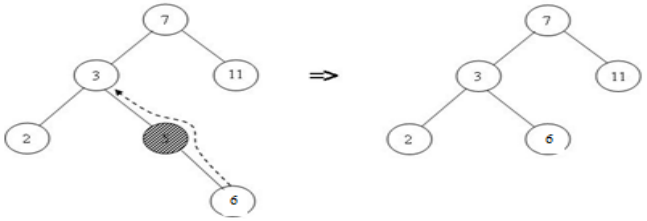
3



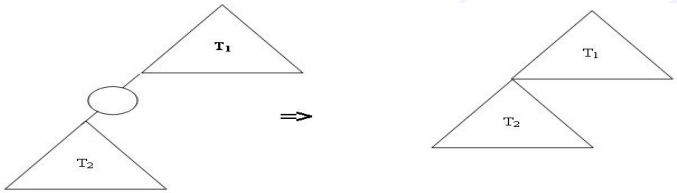

4



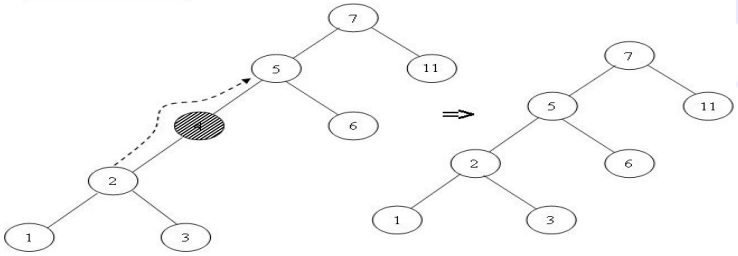



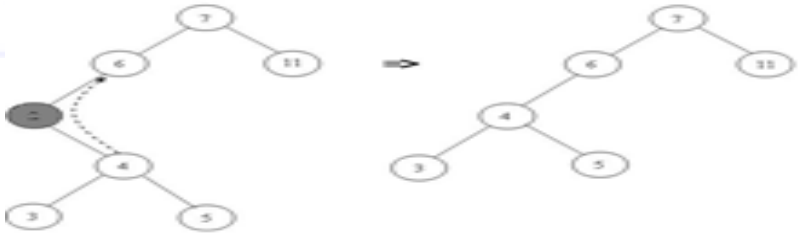




5




6

# Delete from BST - code

```
% delete_bst/3(In_tree, Key_to_del, Out_tree)
[d1] delete_bst(t(Left,Key,Right), K, t(NewLeft,Key,Right)):-
       K<Key,!,
       delete_bst(Left, K, NewLeft).
[d2] delete_bst(t(Left,Key,Right), K, t(Left,Key,NewRight)):-
       K>Key,!,
       delete_bst(Right, K, NewRight).

[d3] delete_bst(nil,              K, nil):-!,
       write(K),write('not found').
[d4] delete_bst(t(nil,K,nil),     K, nil):-!.
[d5] delete_bst(t(nil,K,Right), K, Right):-!.
[d6] delete_bst(t(Left,K,nil),  K, Left):-!.
```

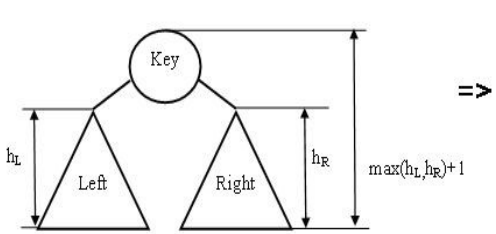
Qs:

- What cases did we cover so far?
- Which clause covers which case? Explain!
- Do we need all clauses so far? Why/why not? Justify

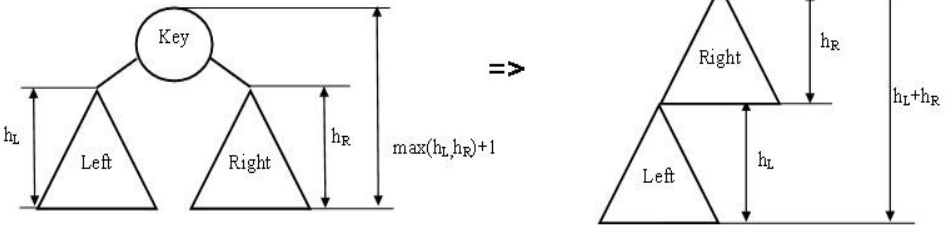
# Delete from BST – code – contd.

- We reached the node containing the key to delete
- Has 2 children = is the root of a subtree =>
  - problem becomes to remove the root of the tree
- How to…? We have alternatives:
  - Either link the subtrees:
    - Easy to apply (time estimate)
    - Disadvantage?

**Sol1**



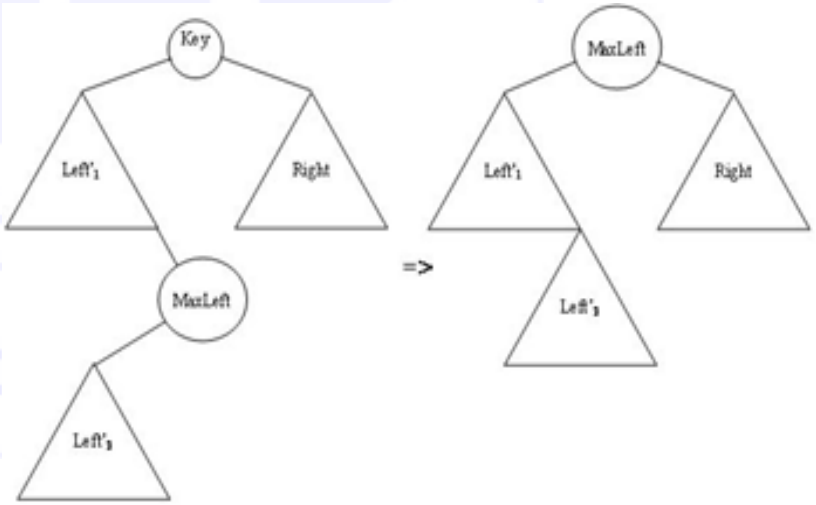

**Sol2**



# Delete from BST – code – contd.

- Or replace problem to solve with a problem you know how to solve!
- Replace INSTEAD the node you want to remove (the root) with another one. How:
  - Find an easy to remove node
  - Place it instead of the root (is possible?)
  - To be possible, that node should be a node in inorder:
    - Either before the key in the root
    - Or after the key in the root
  - So, replace with
    - Predecessor
    - Successor
  - !
  - Are those nodes with the desired property?
  - Justify

**Sol3**



# Delete from BST – code – contd.

```
delete_bst(t(Left,Key,Right), Key, t(NewLeft,MaxLeft,Right)):-!,
        deleteMaxFromTree(Left,MaxLeft,NewLeft).

% deleteMaxFromTree/3(In_tree,Max_key,Out_Tree).
```

- The predicate takes a tree as input `In_tree`, finds the max key `Max_key` (the rightmost node) and removes it from the tree, returning the tree `Out_tree` without that key.

```
deleteMaxFromTree(t(Left,Key,Right), MaxKey, t(Left,Key,NewRight)):-
        deleteMaxFromTree(Right,MaxKey,NewRight).
deleteMaxFromTree(t(Left,MaxKey,nil),MaxKey, Left):-!.
```

- Clause 1 says: as long as the right subtree is not empty, go to the right
- Clause 2 says: if right is `nil`, you found the max key `MaxKey` and `Left` is out tree.
- Do we ever reach second clause? Infinite loop. Clauses are not in the correct order! Let's change it!

# Delete from BST – code – contd.

```
deleteMaxFromTree(t(Left,MaxKey,nil),MaxKey, Left):-!.
deleteMaxFromTree(t(Left,Key,Right), MaxKey, t(Left,Key,NewRight)):-
        deleteMaxFromTree(Right,MaxKey,NewRight).
```

- Now is highly inefficient! We always try the non-useful clause first! Too much!
- Do we really need to have the fact first?
- No, is good to go second (JUSTIFY!).

```
deleteMaxFromTree(t(Left,Key,Right), MaxKey, t(Left,Key,NewRight)):-
        deleteMaxFromTree(Right,MaxKey,NewRight).
deleteMaxFromTree(t(Left,MaxKey,nil),MaxKey, Left):-!.
```

# Delete from BST – complete code

```
% search
delete_bst(t(Left,Key,Right), K, t(NewLeft,Key,Right)):-
        K<Key,!,
        delete_bst(Left, K, NewLeft).
delete_bst(t(Left,Key,Right), K, t(Left,Key,NewRight)):-
        K>Key,!,
        delete_bst(Right, K, NewRight).

% cases of no children or 1 child
delete_bst(nil,K,nil):-!,write(K),write('not found').
delete_bst(t(nil,K,Right), K, Right):-!.
delete_bst(t(Left,K,nil), K, Left):-!.

% case of 2 children
delete_bst(t(Left,Key,Right),Key,t(NewLeft,MaxLeft,Right)):-!,
        deleteMaxFromTree(Left,MaxLeft,NewLeft).

deleteMaxFromTree(t(Left,Key,Right),MaxKey,t(Left,Key,NewRight)):-
        deleteMaxFromTree(Right,MaxKey,NewRight).
deleteMaxFromTree(t(Left,MaxKey,nil),MaxKey,Left):-!.
```

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# Delete from BST – code analysis

- Search phase: O(h)
- Delete (no matter the solution/case chosen): O(h)
- Overall: 2h
- O(h), constant 2. Is it so?
- It is just O(h), constant 1. JUSTIFY!

# Incomplete structures Lists

- Structures ending in variables = instead of the specific empty structure it ends in a variable.
- This IS a list ending in a variable: $L_1$=[1,2,3**|A**]
  - Tail is VARIABLE
  - Can you specify its length?
  - Tail can be anything (including a list, such as [5,6]). BUT in this case, it unifies with the variable and the structure would be a homogeneous one: $L_1$=[1,2,3**,4,5**]
- This IS NOT a list ending in a variable $L_2$=[1,2,3**,B**]
  - Last element variable, can be anything (including a list, such as [4,5]). BUT in this case, it unifies the variable, the structure would be a heterogeneous one: $L_2$=[1,2,3,**[4,5]**]

# Incomplete Lists - Search

```
member(H,[H|_]).
member(H,[_|T]):-
       member(H,T).
```

**[q1]** `?- L=[1,2,3|_],member(2,L).`

`Yes, L=[1,2,3|_]`

**[q2]** `?- L=[1,2,3|_],member(4,L).`

`Yes, L=[1,2,3,4|_]`

- Wrong behavior. Why? How can we fix the issue?

# Incomplete Lists - search

```
member_IL(H,[H|_]):-!.          % item found, stop. For good!
member_IL(H,[_|T]):-            % item not found,
        member_IL(H,T).         % go search in tail
member_IL(_,L):-
        var(L), !,              %reached final free variable
        fail.                   %stop with failure.
```

```
[q1] ?- L=[1,2,3|_],member_IL(2,L).
Yes, L=[1,2,3|_]

[q2] ?- L=[1,2,3|_],member_IL(4,L).
Yes, L=[1,2,3,4|_]
```

- Again, same wrong behavior. How come we didn't fix the issue?

# Incomplete Lists – search – contd.

```
member_IL(_,L):-
        var(L),!,                  % this order: check, cut, fail
        fail.
member_IL(H,[H|_]):-!.             % cut mandatory
member_IL(H,[_|T]):-
        member_IL(H,T).

[q1] ?- L=[1,2,3|_],member_IL(2,L).
Yes, L=[1,2,3|_].                  %exe stops on clause 2

[q2] ?- L=[1,2,3|_],member_IL(4,L).
No.                                %exe stops on clause 1
```

- It NEVER reaches clause #3 in the previous code? Why?
- **Stop conditions** for **incomplete** structures are (i) with **cut** (!) and (ii) ALWAYS **come first**. Otherwise, it is as if they are missing!
  - Starts with failure condition(s). It MUST be explicit (NEVER default).
  - Continues with the successful stop condition(s).

# Incomplete Lists – insert

```
insert_IL(X,L):-
        var(L),                 % reached the final free variable?
        !,                      % stop backtracking
        L=[X|_].                % update the list with the new added item
insert_IL(H,[H|_]):-!.          % item found, and don't allow backtracking
insert_IL(H,[_|T]):-
        insert_IL(H,T).
```

**[q1]** `?- L=[1,2,3|_], insert_IL(4,L).`

`Yes, L=[1,2,3,4|_].`

**[q2]** `?- L=[1,2,3|_], insert_IL(3,L).`

`Yes, L=[1,2,3|_].`

- q1: pure insert behavior (stop on clause 1)
- q2: member behavior (stop on clause 2)

# Incomplete Lists – insert – contd.

- We don't really need clause 1. Clause 2 covers it (default).

```
insert_IL(H,[H|_]):-!.
insert_IL(H,[_|T]):-
       insert_IL(H,T).
```

**[q1]** `?- L=[1,2,3|_], insert_IL(4,L).`

`Yes, L=[1,2,3,4|_].`

Has pure insert behavior. Clause 1 behaves as:

```
insert_IL(X,L):-
     var(L),!,L=[X|_].
```

**[q2]** `?- L=[1,2,3|_], insert_IL(3,L).`

`Yes, L=[1,2,3|_].`

Has member behavior. Clause 1 behaves as:

```
insert_IL(X,[H|_]):-X=H, !.      % member(X,[X|_]):-!.
```

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

# Conclusions

- Trees – basic operations
- Incomplete lists